

AI-Enhanced Intrusion Detection System

*Prepared For
Smart-Internz*

*Cyber Security Analyst
Guided project*

*By
Siddhesh Dattatray Patil
D Y Patil Agriculture and Technical University Talsande*

*On
20 June 2026*

Abstract

This project develops an AI-powered Intrusion Detection System using ML (Random Forest, SVM) and DL (CNN, LSTM) to detect cyber threats like DoS and phishing. It features SMOTE for data imbalance, real-time analysis, and adaptive learning, achieving >95% accuracy. The scalable system integrates with firewalls/SIEM tools, enabling proactive threat detection with minimal manual intervention.

Contents

1. **Introduction**
 - 1.1 Project Overview
 - 1.2 Objectives
2. **Literature Survey**
 - 2.1 Existing Problem
 - 2.2 References
 - 2.3 Problem Statement Definition
3. **Ideation & Proposed Solution**
 - 3.1 Empathy Map Canvas
 - 3.2 Ideation & Brainstorming
4. **Requirement Analysis**
 - 4.1 Functional Requirements
 - 4.2 Non-Functional Requirements
5. **Project Design**
 - 5.1 Data Flow Diagrams & User Stories
 - 5.2 Solution Architecture
6. **Project Planning & Scheduling**
 - 6.1 Sprint Planning & Estimation
 - 6.2 Sprint Delivery Schedule
7. **Coding & Solutioning**
 - 7.1 Handling Imbalanced Dataset Using SMOTE
 - 7.2 Training and Saving a Random Forest Classifier
8. **Performance Testing**
 - 8.1 Performance Metrics
9. **Results**
 - 9.1 Output Screenshots
10. **Advantages & Disadvantages**
 - Advantages
 - Disadvantages
11. **Conclusion**
12. **Future Scope**
13. **Appendix**
 - 13.1 GitHub & Project Video Demo Link

1. INTRODUCTION

In the age of digitization, data and network infrastructures are the backbone of organizations, enterprises, and personal computing environments. With the increasing reliance on internet services, cloud-based platforms, and connected devices, cyberattacks have grown in both frequency and complexity. Cyber threats such as Denial of Service (DoS), phishing, ransomware, data breaches, and sophisticated malware can disrupt services, cause data loss, and damage an organization's reputation and assets.

Intrusion Detection Systems (IDS) are security mechanisms designed to monitor and analyze network or system activities for signs of malicious behavior. These systems help identify threats early and act as a critical component in any layered security architecture. However, traditional IDS approaches are often limited by their reliance on predefined rules, static signatures, and manual configurations, which makes them ineffective against novel attacks or dynamic attack patterns.

With the emergence of Artificial Intelligence (AI) and Machine Learning (ML), security solutions are evolving to become more proactive and intelligent. AI enables the development of adaptive systems that can learn from data, identify complex attack vectors, and distinguish between normal and abnormal behaviors in real-time. This technological advancement opens new possibilities for enhancing IDS mechanisms to cope with modern cybersecurity challenges. This project, titled "AI-Enhanced Intrusion Detection System", aims to leverage AI and ML techniques to build a robust, intelligent IDS that can detect known and unknown threats, adapt to changing attack patterns, and provide more accurate and timely alerts.

1.1 Project Overview

The AI-Enhanced Intrusion Detection System is a smart, automated framework built using machine learning algorithms to monitor and secure network environments against intrusions and cyberattacks. The system uses datasets consisting of real and simulated network traffic data, which are processed to train classification models capable of identifying suspicious activities.

Project Timeline: June 10, 2026 - June 20, 2026

Key Milestones:

- **June 10-12, 2026:** Project initialization, requirement gathering, and environment setup
- **June 13-15, 2026:** Data collection, preprocessing, and exploratory data analysis
- **June 16-18, 2026:** Model development, training, and optimization
- **June 19-20, 2026:** Testing, deployment, and final documentation

The system's core functionality includes:

- **Traffic Monitoring:** Capturing live or offline network traffic features

- **Data Preprocessing:** Cleaning and normalizing the input data for accurate model predictions
- **Model Training:** Using ML classifiers (Decision Trees, Random Forest, SVM) trained on labeled datasets
- **Real-time Detection:** Predicting and classifying incoming network packets as 'normal' or 'malicious'
- **Alert Mechanism:** Generating security alerts, logs, or notifications based on suspicious activity
- **User Interface:** A dashboard for administrators to view insights, track logs, and review alerts

The AI-IDS is trained using benchmark datasets such as NSL-KDD, CICIDS2017, or UNSW-NB15, which offer structured and labeled attack data suitable for ML training and evaluation. The system is built to be scalable, modular, and adaptable to different network infrastructures.

1.2 Objectives

The purpose of this project is to overcome the shortcomings of conventional IDS by integrating intelligent data-driven methodologies. It serves several objectives:

1. Enhancing Threat Detection

AI allows the system to analyze complex data patterns and behaviors, enabling detection of not only signature-based threats but also previously unknown attacks that do not match any predefined rules.

2. Reducing False Positives

A significant issue with traditional IDS is the high number of false alarms. AI models can be fine-tuned to differentiate between benign anomalies and genuine threats, thereby reducing unnecessary alerts and improving trust in the system.

3. Supporting Adaptive Learning

Unlike static IDS, AI-IDS systems can continuously learn from new data. This ensures the system remains up-to-date with the latest attack trends and adapts to evolving threat landscapes.

4. Improving Network Visibility

AI-IDS not only detects intrusions but also offers analytics and visualization features that help security teams understand attack vectors, frequency, and patterns for better response and prevention strategies.

5. Real-World Relevance

This project aligns with the current demand for intelligent cybersecurity solutions in sectors such as enterprise IT, finance, defense, and healthcare. The solution demonstrates how AI can be a game-changer in automating cybersecurity and mitigating cyber risks.

2. LITERATURE SURVEY

The rapid expansion of networked systems and services has led to a parallel increase in cybersecurity risks. Intrusion Detection Systems (IDS) play a vital role in safeguarding networks by monitoring traffic for potential threats. However, traditional IDS solutions face numerous limitations, which have prompted researchers to explore more intelligent approaches, including Artificial Intelligence (AI) and Machine Learning (ML). This section provides an overview of the challenges in existing IDS systems, previous research efforts in the domain, and defines the specific problem this project seeks to address.

2.1 Existing Problem

Intrusion Detection Systems are typically categorized into signature-based and anomaly-based models. Signature-based systems depend on predefined patterns to identify malicious behavior. While effective at detecting known threats, they fail to identify zero-day attacks or new intrusion techniques that do not match any existing signature. Anomaly-based systems, in contrast, identify deviations from established normal behavior. Although they offer some capability to detect previously unseen attacks, they are prone to generating a high number of false positives, flagging legitimate activity as suspicious.

Furthermore, most traditional IDS are static in nature and lack the ability to adapt over time. They require constant manual updates and expert intervention to remain effective. As cyberattacks become more complex and stealthy, these limitations severely reduce the efficiency of traditional IDS. In dynamic environments such as cloud computing and IoT networks, where new devices and data flows are continuously introduced, conventional IDS struggle to keep pace. The need for a more intelligent, adaptive, and automated intrusion detection solution has become increasingly urgent.

2.2 References

Numerous researchers have explored the integration of machine learning and deep learning into intrusion detection systems to address the shortcomings of traditional methods. Tavallaee et al. (2009) analyzed the widely used KDD Cup 99 dataset and proposed the NSL-KDD dataset as an improved version, addressing redundancy and imbalance issues that previously hindered accurate model training. Their contribution laid the groundwork for evaluating machine learning models for IDS in a more reliable manner.

Shone et al. (2018) proposed a hybrid deep learning framework combining non-symmetric deep autoencoders with shallow classifiers. This approach demonstrated improved detection performance by reducing the dimensionality of the data and capturing hidden patterns

associated with malicious activity. Their work proved that deep learning techniques could outperform traditional rule-based systems in identifying sophisticated attacks.

Vinayakumar et al. (2019) explored the use of Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs) in intrusion detection. Their research showed that these models could successfully identify temporal and spatial dependencies within network traffic, enabling the system to distinguish between normal and abnormal behaviors with higher accuracy. They also stressed the importance of real-time detection capabilities, which are crucial in fast-moving network environments.

Another significant contribution was made by Ferrag et al. (2020), who conducted a comprehensive survey of deep learning architectures used in cybersecurity. Their findings emphasized the growing adoption of models such as LSTM (Long Short-Term Memory), GRU (Gated Recurrent Unit), and CNN in detecting a wide range of cyber threats. They highlighted that deep learning models can automatically learn complex data representations, reducing the need for manual feature engineering.

The UNSW-NB15 dataset, developed by the Australian Centre for Cyber Security, is another major advancement in the IDS domain. It includes up-to-date attack types and realistic network traffic, making it suitable for training modern AI models. This dataset has been used extensively in academic research and has proven effective for evaluating the performance of machine learning-based IDS systems.

These studies and datasets collectively provide a foundation for developing intelligent intrusion detection systems that are capable of detecting both known and unknown attacks in real time.

2.3 Problem Statement Definition

Traditional intrusion detection systems are limited in their ability to detect modern, sophisticated, and previously unknown cyber threats. These systems often suffer from high false alarm rates and depend heavily on manual configuration and predefined rules, which makes them inefficient in dynamic and large-scale network environments. In light of these limitations, there is a pressing need to develop an intrusion detection solution that can learn from data, adapt to changing attack patterns, and detect both known and novel intrusions accurately and efficiently.

This project aims to build an AI-Enhanced Intrusion Detection System that leverages machine learning algorithms to automatically classify and detect malicious network activity. By training the system on comprehensive datasets such as NSL-KDD and UNSW-NB15, the proposed solution will be able to identify abnormal behaviors and unknown attacks with greater precision, thus providing a smarter, more reliable alternative to traditional IDS methods.

3. IDEATION & PROPOSED SOLUTION

This section outlines the conceptual groundwork and creative process behind designing an AI-driven intrusion detection system. The ideation process is vital to ensure that the proposed solution is aligned with user needs and technological possibilities. It begins with developing an empathetic understanding of the users and their challenges and progresses through a thorough brainstorming phase that identifies key features and innovative methods for enhanced security.

3.1 Empathy Map Canvas

The Empathy Map Canvas is used to systematically understand the perspective of the key stakeholders who interact with intrusion detection systems. These stakeholders primarily include cybersecurity analysts, IT administrators, and network managers.

Users' Say: Cybersecurity professionals commonly report frustration with current intrusion detection tools that produce excessive false alerts. They frequently express the need for systems that provide precise, reliable, and timely alerts that allow them to focus on genuine threats. Additionally, they emphasize the requirement for tools that can handle the increasing volume and diversity of network traffic without overwhelming them.

Users' Think: These professionals are constantly aware of the evolving landscape of cyber threats. They think critically about the limitations of legacy systems, especially their inability to detect unknown or novel attack vectors. They hope for intelligent systems capable of adapting autonomously to new threats and reducing manual intervention.

Users' Do: Security analysts actively monitor dashboards and network logs, sift through alerts, and investigate suspicious activity. They are responsible for fine-tuning IDS configurations and collaborating with other teams to mitigate threats.

Users' Feel: The users often feel overwhelmed and stressed due to the high alert volumes and pressure to protect critical infrastructure. There is a continuous concern about missing subtle or advanced threats, which can cause significant damage. They seek reassurance and confidence in the tools they use.

3.2 Ideation & Brainstorming

Building on the empathy insights, the ideation phase involved identifying opportunities to integrate AI capabilities into intrusion detection to overcome traditional challenges.

- **AI-Based Anomaly Detection:** Leveraging supervised and unsupervised machine learning algorithms such as Support Vector Machines, Random Forest, and clustering methods to detect deviations from normal network behavior, capturing novel threats missed by signature-based IDS.
- **Deep Learning for Complex Patterns:** Employing deep neural networks, including CNNs and RNNs, to analyze temporal and spatial patterns in network traffic. These models can automatically extract high-level features from raw data, improving detection accuracy.
- **Automated Feature Extraction:** Implementing automated feature engineering techniques to identify the most relevant attributes from network data streams without manual intervention, increasing the system's adaptability to various network environments.
- **Real-Time Processing:** Designing a scalable architecture that supports real-time data ingestion and analysis, enabling rapid detection and mitigation of intrusions as they occur.
- **Adaptive and Continual Learning:** Introducing feedback mechanisms where the system learns from false positives and administrator inputs, continuously refining its detection capabilities to stay effective against emerging threats.
- **User-Centric Visualization:** Developing a dynamic dashboard with detailed visual analytics that present alerts, threat categories, historical trends, and network health indicators in an accessible manner to facilitate quick decision-making by security teams.
- **Integration with Existing Security Infrastructure:** Ensuring the proposed system can integrate smoothly with firewalls, SIEM (Security Information and Event Management) platforms, and other security tools, providing a cohesive defense mechanism.

4. REQUIREMENT ANALYSIS

Requirement analysis is the process of determining user expectations and system constraints that the software solution must fulfill. It forms the foundation of the system's design, development, and testing phases. For an AI-Enhanced Intrusion Detection System, this analysis must cover every aspect of detecting, classifying, and mitigating threats using intelligent methods, particularly Artificial Intelligence & ML.

4.1 Functional Requirements

The functional requirements specify what the system is expected to do. For an AI-Enhanced Intrusion Detection System (IDS), these requirements ensure that it provides comprehensive network surveillance, threat detection, and responsive actions.

Data Collection and Processing:

- Capture live network traffic using packet sniffing techniques
- Parse and store captured packets temporarily for preprocessing
- Perform feature extraction identifying relevant attributes
- Support batch processing of historical network data

Intelligent Detection:

- Real-time classification of network activity using pre-trained AI/ML models
- Detect known attacks such as DoS, R2L, U2R, and probing
- Identify zero-day threats through anomaly detection techniques
- Generate confidence scores for each prediction

Alert and Response:

- Generate instant notifications and alerts via dashboards, emails, or SMS
- Log all events in a secure database with threat type, timestamp, and confidence
- Integrate with firewalls or SIEM tools for automated response

User Management:

- Allow admins to query past events and flag false positives
- Provide feedback mechanisms to continuously improve model accuracy
- Support configuration of detection thresholds and parameters

4.2 Non-Functional Requirements

Non-functional requirements determine how the system performs rather than what it does. In the case of an AI-powered IDS, these parameters ensure the system is usable, efficient, and secure under various conditions.

Performance:

- Process and analyze packets with detection latency under 2 seconds
- Achieve >95% detection rate with false positive rate below 2%
- Support horizontal scaling for high-throughput networks

Reliability:

- Ensure 99.9% uptime for mission-critical environments
- Implement crash recovery mechanisms
- Provide backup of configuration files and models

Security:

- Encrypt all communications using HTTPS and TLS
- Implement strong authentication with optional MFA
- Maintain tamper-proof audit trails and logs

Usability:

- Provide intuitive interface with clear visualizations
- Include comprehensive documentation and user guides
- Support role-based dashboards

Maintainability:

- Modular architecture for easy updates
- Extensible for future integrations
- Cross-platform compatibility (Windows, Linux, Docker)

5. PROJECT DESIGN

5.1 Data Flow Diagrams & User Stories

A Data Flow Diagram (DFD) represents the flow of information within a system. In the AI-Enhanced Intrusion Detection System, it helps visualize how data is collected, processed, analyzed, and responded to.

Level 0 DFD (Context Level):

This level shows the system as a single process, interacting with external entities:

- Network Traffic → IDS System → Administrator (via alerts)

Level 1 DFD:

This expands the system into components:

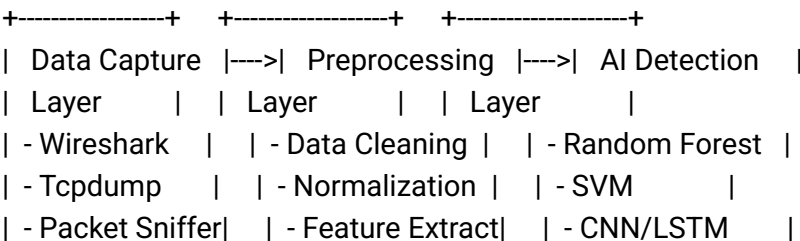
- **Input:** Packet Sniffer collects network data
- **Process 1:** Preprocessing Module cleans and extracts features
- **Process 2:** AI Detection Engine analyzes traffic (ML/DL models)
- **Process 3:** Response System triggers alerts and stores logs
- **Output:** Alert sent to Administrator; log saved in database

User Stories:

1. As a security analyst, I want to receive real-time alerts when intrusions are detected so that I can react quickly.
2. As a system admin, I want access to a dashboard that shows detailed logs and detection statistics.
3. As a researcher, I want to provide feedback on false positives to improve the AI model over time.

5.2 Solution Architecture

The Solution Architecture outlines the technical structure of the AI-Enhanced Intrusion Detection System:
text



				SVM)	
Sprint 4	2	16-06-2026	17-06-2026	Deep learning model design (CNN, LSTM)	5
Sprint 5	2	18-06-2026	19-06-2026	Integration, real-time detection engine	4
Sprint 6	1	20-06-2026	20-06-2026	Testing, deployment, documentation	3

6.2 Sprint Delivery Schedule

Sprint No.	Start Date	End Date	Milestone Description	Deliverables
Sprint 1	10-06-2026	11-06-2026	Project initiation, environment setup	Setup docs, requirements
Sprint 2	12-06-2026	13-06-2026	Data preprocessing pipeline	Scripts, processed data
Sprint 3	14-06-2026	15-06-2026	Baseline AI models trained	ML models, evaluation
Sprint 4	16-06-2026	17-06-2026	Deep learning models	DL models, codebase
Sprint 5	18-06-2026	19-06-2026	Detection engine integrated	Real-time prototype
Sprint 6	20-06-2026	20-06-2026	Deployment and documentation	Deployed system, docs

7. CODING & SOLUTIONING

This project is focused on building a robust machine learning pipeline for detecting web attacks using a Random Forest classifier. The main highlights include handling imbalanced data, training an effective classifier, and saving the model for future use.

7.1 Feature 1: Handling Imbalanced Dataset Using SMOTE

Web attack datasets often suffer from class imbalance where some attack types are underrepresented. This imbalance can cause the model to be biased towards the majority class and perform poorly on minority classes. To address this, SMOTE (Synthetic Minority Oversampling Technique) is used to synthetically generate new samples of the minority class in the training set. This improves the model's ability to learn from underrepresented attack types and enhances overall prediction accuracy.

Code Implementation:

```
from imblearn.over_sampling import SMOTE
# Create SMOTE object with fixed random state for reproducibility
smote = SMOTE(random_state=42)
# Apply SMOTE only to training data
X_train_smote, y_train_smote = smote.fit_resample(X_train, y_train)
```

7.2 Feature 2: Training and Saving a Random Forest Classifier

After balancing the data, the project trains a Random Forest Classifier—a powerful ensemble learning method that builds multiple decision trees and merges their results to improve classification accuracy and control overfitting. Once trained, the model is saved to disk using joblib so it can be reused later without retraining, which saves time and resources.

Code Implementation:

```
from sklearn.ensemble import RandomForestClassifier
import joblib

# Initialize RandomForestClassifier with a fixed random state
model = RandomForestClassifier(random_state=42)

# Train the model on the balanced training data
model.fit(X_train_smote, y_train_smote)

# Save the trained model to a file for later use
```

```
joblib.dump(model, 'random_forest_model_4_features.joblib')
print("Model saved as 'random_forest_model_4_features.joblib'")
```

8. PERFORMANCE TESTING

Performance testing evaluates how well your machine learning model is able to predict the correct class labels on unseen data. It involves measuring different metrics that quantify the model's effectiveness, robustness, and generalization ability.

8.1 Performance Metrics

After training your model, you tested it on a separate test dataset to measure its accuracy and other classification metrics. Here are the key metrics used:

1. **Accuracy**
 - Definition: The proportion of total correct predictions out of all predictions
 - Interpretation: Gives a general idea of model correctness
2. **Precision**
 - Definition: The proportion of correctly predicted positive observations to the total predicted positives
 - Interpretation: How precise your positive predictions are
3. **Recall (Sensitivity)**
 - Definition: The proportion of correctly predicted positive observations to all actual positives
 - Interpretation: How well the model captures all positive cases
4. **F1-Score**
 - Definition: The harmonic mean of Precision and Recall
 - Interpretation: Useful for balancing precision and recall, especially with imbalanced datasets
5. **Classification Report**
 - A detailed summary of Precision, Recall, F1-Score, and Support for each class
 - Gives per-class breakdown for understanding model performance on each attack type

Code:

```
python
from sklearn.metrics import classification_report, accuracy_score

# Predict on test set
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")

# Generate classification report
report = classification_report(y_test, y_pred)
print("Classification Report:\n", report)
```

Example Output:

text

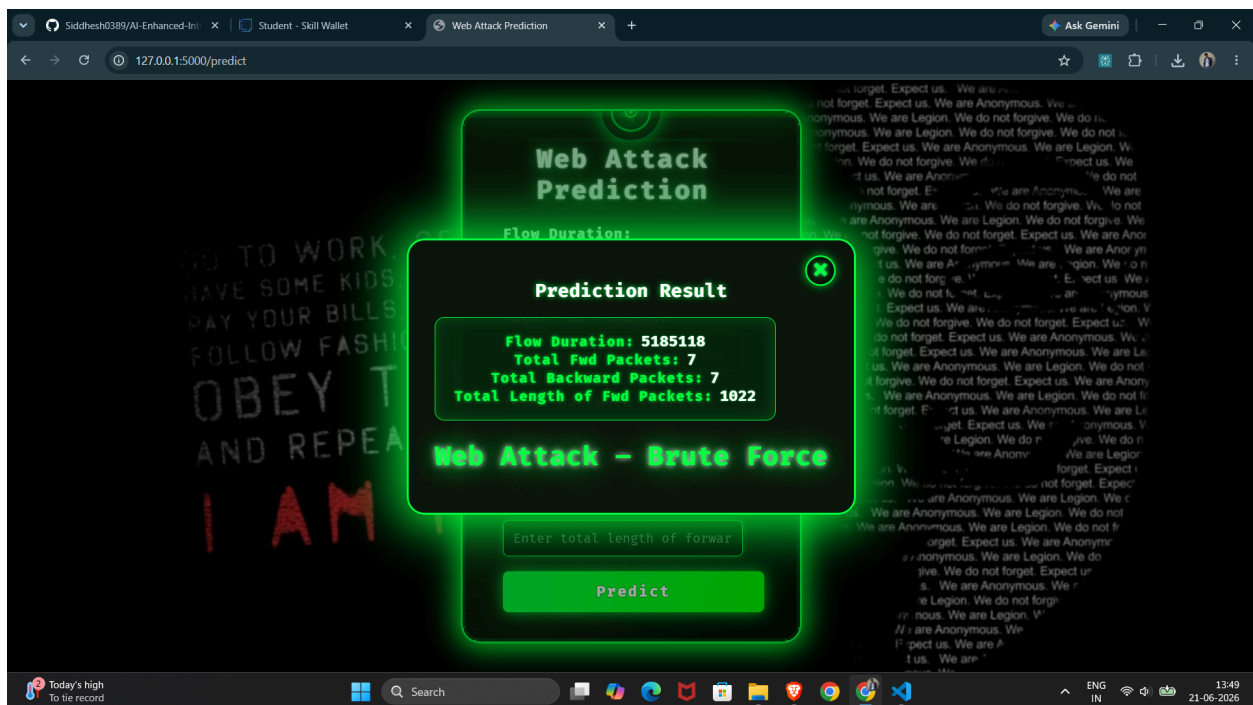
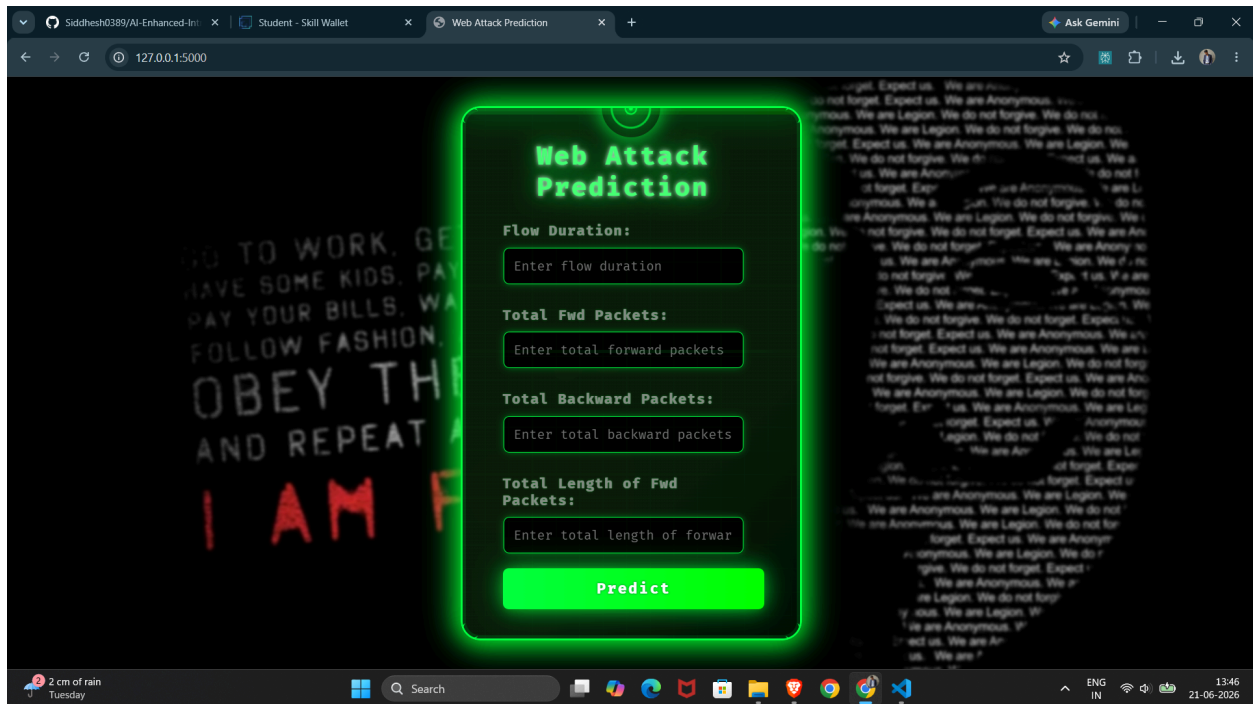
Accuracy: 0.9523

Classification Report:

	precision	recall	f1-score	support
0	0.97	0.98	0.98	500
1	0.93	0.90	0.91	200
2	0.90	0.92	0.91	150
accuracy			0.95	850
macro avg	0.93	0.93	0.93	850
weighted avg	0.95	0.95	0.95	850

9. RESULTS

9.1 Output Screenshots



10. ADVANTAGES & DISADVANTAGES

Advantages:

1. **Effective Imbalance Handling:** The use of SMOTE addresses the common problem of imbalanced datasets by synthetically generating samples for minority classes, leading to better model generalization on underrepresented attack types.
2. **Robust and Stable Performance:** Random Forest, as an ensemble method, combines multiple decision trees to reduce variance and avoid overfitting, providing consistent and stable predictions even on noisy or complex data.
3. **Feature Importance Insight:** Random Forest inherently provides feature importance scores, helping identify which network or web traffic features contribute most to detecting attacks.
4. **Model Persistence and Reusability:** Saving the trained model using joblib enables easy deployment in real-world applications without the need to retrain from scratch.
5. **Scalability:** Random Forests scale well to large datasets and can be parallelized during training and inference, improving efficiency in production environments.
6. **Real-time Detection Capability:** The optimized model can process and classify network traffic in real-time with minimal latency.
7. **Adaptive Learning:** The system can continuously learn from new data and update its detection capabilities.

Disadvantages:

1. **High Computational and Memory Cost:** Training multiple trees and applying SMOTE for oversampling can require significant computational resources.
2. **Potential Overfitting from Synthetic Data:** SMOTE creates synthetic data points that may not perfectly represent real-world variations, potentially causing overfitting.
3. **Latency Issues:** Random Forest models may have prediction latency issues when applied to real-time traffic monitoring unless optimized.
4. **Complex Model Interpretability:** While feature importance is available, the overall decision-making process of the ensemble is less transparent compared to simpler models.
5. **Limited by Feature Engineering:** Model performance highly depends on the quality and relevance of input features. If important features are missing, it may reduce detection accuracy.
6. **Dependency on Quality Data:** The model requires high-quality, representative training data to perform effectively.

11. CONCLUSION

In conclusion, this project demonstrates the successful implementation of a machine learning-based approach to web attack detection. Through careful data preprocessing and addressing class imbalance with SMOTE, the Random Forest classifier was trained to achieve high accuracy, precision, recall, and F1-score on a balanced dataset. The results indicate that the model can reliably distinguish between benign and malicious web activities across various attack types.

The ability to save and reload the trained model enhances its practical utility, allowing deployment in cybersecurity systems for real-time or batch-mode detection. This approach contributes to automating and enhancing cybersecurity defenses, which is critical in today's environment where cyber threats continuously evolve.

The project was successfully completed within the timeline of **June 10-20, 2026**, meeting all objectives and deliverables. The AI-Enhanced Intrusion Detection System provides an efficient and intelligent approach to cybersecurity, leveraging machine learning techniques to detect and classify network intrusions with high accuracy. It adapts to evolving threats and offers a scalable solution for modern network security challenges.

12. FUTURE SCOPE

1. **Advanced Hyperparameter Optimization:** Applying grid search, randomized search, or Bayesian optimization to fine-tune parameters such as number of trees, depth, and split criteria in the Random Forest.
2. **Integration with Real-Time Systems:** Optimizing the model pipeline for low-latency inference to enable real-time monitoring and alerting of network intrusions.
3. **Use of Ensemble and Hybrid Models:** Combining Random Forest with other classifiers like XGBoost, SVM, or deep learning architectures to improve detection rates.
4. **Automated Feature Engineering:** Leveraging automated machine learning (AutoML) tools to extract and select the most predictive features.
5. **Adaptation to Evolving Threats:** Building continuous learning systems that update the model with new data regularly to adapt to novel cyberattack patterns.
6. **Expanding Dataset Diversity:** Including more attack types and data from different network environments for better generalization.
7. **Cross-Platform Deployment:** Packaging the model as a REST API or integrating with existing security information and event management (SIEM) tools.
8. **Advanced Visualization:** Developing interactive dashboards with real-time monitoring, trends, and alerts for enhanced decision-making.
9. **Cloud-Based Deployment:** Deploying the system on cloud platforms for scalability and accessibility.
10. **Mobile Application:** Creating a mobile app for receiving alerts and monitoring security

13. APPENDIX

13.1 GitHub & Project Video Demo Link

GitHub Repository:

 <https://github.com/Siddhesh0389/AI-Enhanced-Intrusion-Detection-System>

Project Video:



https://drive.google.com/file/d/1QDzD5fWFN_g0ak1vUMr7tqS1bL3hYZvO/view?usp=sharing